

UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA



Project: Parser & SDT (2nd Submission)

Course: Compiladores

Group 5

Semester: 2027-I

Team 04

Blancas Díaz Isaías
Hernández Flores Alan Uziel
Martínez Sánchez Hans
Ramírez Terán Emily
Valdez Subirach Sebastián

Public Repository:

`https://github.com/HansMartza/unam.fi.compilers.g5.04.git`

November 10th, 2026

Contents

1. Introduction **2**

1.1. Problem Statement 2

1.2. Motivation 2

1.3. Objectives 2

2. Theoretical Framework **3**

2.1. Context-Free Grammars (CFG) and BNF Notation 3

2.2. Top-Down Parsing and LL(1) Recursive Descent 3

2.3. Syntax-Directed Translation (SDT) and Attribute Propagation 4

2.4. Symbol Table and Static Semantic Verification (Type Checking) 4

3. Development **5**

3.1. General System Architecture 5

3.2. Definitive Grammar Specification and SDT Rules 5

3.3. Algorithmic Description of Main Components and Classes 7

3.4. Theoretical Justification of Output Scenarios and Error Handling . . . 7

4. Results **8**

4.1. Scenario 1: Successful Syntactic Recognition and Semantic Verification 8

4.2. Scenario 2: Syntactic Error Detection (*Parsing Error*) 9

4.3. Scenario 3: Semantic Error Detection (*Type Mismatch*) 9

5. Conclusions **10**

1. Introduction

1.1. Problem Statement

In constructing a compiler's *Front-End*, following the lexical analysis phase in which the source program's character string is converted into a structured list of lexical tokens (*tokens*), the next stage consists of verifying the syntactic validity of the sequence and ensuring that it satisfies the scope and type compatibility rules of the target language.

The problem addressed in this second submission consists of designing and implementing a top-down parser (**Recursive Descent LL(1) Top-Down Parser**) and a **Syntax-Directed Translator (SDT)** in Python. The system processes the tokens emitted by the lexical analyzer (`KEYWORD`, `IDENTIFIER`, `OPERATOR`, `CONSTANT`, `DECIMAL`, `LITERAL`, `PUNCTUATION`), builds an Abstract Syntax Tree (AST), and executes the corresponding semantic rules, emitting accurate diagnostics with line and column numbers.

1.2. Motivation

The top-down recursive descent paradigm stands out for its absolute algorithmic transparency and its direct correspondence between formal language theory and code structure. By designing an LL(1) grammar—left-factored and free of left recursion—each non-terminal variable translates into a deterministic procedural method that decides which production to expand by inspecting a single lookahead token (*lookahead*).

Furthermore, decoupling the validation of **form** (responsibility of the Parser) from the verification of **meaning and types** (responsibility of the SDT in the `SemanticAnalyzer` class) simplifies the grammar rules, achieving maintainable, elegant code that is 100% faithful to the development standards required by the course.

1.3. Objectives

- **General Objective:**
  Build an integrated Top-Down LL(1) Parsing and Syntax-Directed Translation (*SDT*) module in Python that formally validates the structure and static semantics of source code from the *tokens* generated by the Submission 1 *lexer*.
- **Specific Objectives:**
  1. Formalize the Context-Free Grammar in BNF notation, composed of 24 deterministic productions without left recursion.
  2. Implement the parser architecture in Python, integrating the `ParserTreeBuilder` class for AST construction and the `SemanticAnalyzer` class for type checking (*Type Checking*) and operations.

3. Refine error handling through the `errorHandler` class, including precise line and column reporting and support for files with Windows line endings.
4. Demonstrate strict compliance with the three standard output scenarios (`Parsing Success! SDT Verified!`, `Parsing error... SDT error...`, and `Parsing Success! SDT error...`) with their corresponding theoretical justifications.

---

2. Theoretical Framework

2.1. Context-Free Grammars (CFG) and BNF Notation

A Context-Free Grammar is formalized through the 4-tuple $G = (V_N, V_T, P, S)$, where:

- V_N is the finite set of non-terminal variables or abstract syntactic concepts (`Program`, `Statement`, `Declaration`, `Expresion`, `Termino`, `Factor`).
- V_T is the set of terminal symbols or indivisible *tokens* delivered by the scanner (`int`, `float`, `string`, `print`, `id`, `constant`, `decimal`, `literal`, `=`, `+`, `-`, `*`, `/`, `%`, `(`, `)`, `;`).
- P is the set of production rules of the form $A \to \alpha$, with $A \in V_N$ and $\alpha \in (V_N \cup V_T)^*$.
- $S \in V_N$ is the start symbol of the grammar (`Program`).

Backus-Naur Form (BNF) allows describing this structure cleanly. It is called "context-free" because the substitution of A by α is performed independently of the context surrounding A in the input string.

2.2. Top-Down Parsing and LL(1) Recursive Descent

Top-down parsing builds the syntax derivation tree starting from the root (start symbol S) down to the leaves (terminal symbols), applying leftmost derivations (*leftmost derivation*).

The **LL(1)** classification indicates that the parser reads the input from left to right (**L**eft-to-right), applies a leftmost derivation (**L**eftmost), and requires a single lookahead token (**1** symbol of lookahead). For a grammar to be LL(1), it must satisfy two indispensable conditions:

1. **Absence of Left Recursion:** No production may take the form $A \rightarrow A\alpha$, as this would cause an infinite loop in the recursive procedure.
2. **Left Factoring:** The bodies of alternatives for the same non-terminal must begin with disjoint sets of terminals (disjoint `FIRST` sets), ensuring that the current token unambiguously determines the rule to expand.

2.3. Syntax-Directed Translation (SDT) and Attribute Propagation

Syntax-Directed Translation (SDT) is a technique that associates attributes with grammar symbols and executable actions (procedural code) with production rules. Attributes fall into two categories:

- **Synthesized Attributes:** Values calculated at a parent node from the attributes of its child nodes (information flows upward through the tree).
- **Inherited Attributes:** Values transmitted from a parent or left-sibling node down to a child node (information flows downward or horizontally).

In this project, semantic evaluation is implemented via a depth-first traversal (*Visitor*) over the Abstract Syntax Tree (AST), where the types and values of factors are synthesized from leaf nodes (`ValueNode`) up to binary operations (`BinaryOpNode`) and declaration statements (`DeclarationNode`).

2.4. Symbol Table and Static Semantic Verification (Type Checking)

The **Symbol Table** is the associative data structure (dictionary/hash) responsible for storing program identifiers, recording their name, strict type (`int`, `float`, `string`), and position.

During the static semantic phase, the analyzer performs two fundamental checks:

1. **Scope and Registration Verification:** Ensuring that used variables exist and are not redundantly declared.
2. **Type Checking:** Guaranteeing compatibility between the declared type for a variable and the type resulting from evaluating the assigned expression (e.g., preventing storing floating-point values or strings inside an integer container).

---

3. Development

3.1. General System Architecture

Following software architecture standards, the application is divided into specialized components:

Source Code (`.txt`)
↓
`Lexer` ← *Unified regular expressions (`re.finditer`)*
↓ *Token Stream:* `[Token(KEYWORD, 'int'), Token(IDENTIFIER, 'x'), ...]`
`ParserTreeBuilder` ← *Top-Down LL(1) Recursive Descent (Validates SYNTAX & builds AST)*
↓ *Abstract Syntax Tree (AST Root:* `ProgramNode`*)*
`SemanticAnalyzer` ← *Traverses AST, verifies TYPES (SDT) and Symbol Table*
↓
`errorHandler` ← *Generates final output report with flags and diagnostics*

3.2. Definitive Grammar Specification and SDT Rules

The formal specification consists of 24 production rules expressed in deterministic LL(1) BNF notation and their corresponding semantic rules for evaluating synthesized and inherited attributes.

Grammar $G = (V_N, V_T, P, S)$

- **Start Symbol (S):** `Program`
- **Terminal Symbols (V_T):** `int`, `float`, `string`, `print`, `id`, `constant`, `decimal`, `literal`, `=`, `+`, `-`, `*`, `/`, `%`, `(`, `)`, `;`
- **Non-Terminal Symbols (V_N):** `Program`, `StmtList`, `Statement`, `Declaration`, `PrintStatement`, `Type`, `Var`, `Expresion`, `Expresion'`, `Termino`, `Termino'`, `Factor`

Grammatical Production Table and Semantic Actions (SDT)

#	Grammatical Production (LL1)	Semantic Rule / SDT Action
1	⟨Program⟩ →⟨Statement⟩ ⟨StmtList⟩	{ Program = Statement StmtList }
2	⟨StmtList⟩ →⟨Statement⟩ ⟨StmtList1⟩	{ StmtList = Statement StmtList1 }
3	⟨StmtList⟩ →ε	{ StmtList = ε }
4	⟨Statement⟩ →⟨Declaration⟩	{ Statement = Declaration.exp }
5	⟨Statement⟩ →⟨PrintStatement⟩	{ Statement = PrintStatement.val }
6	⟨Declaration⟩ →⟨Type⟩ ⟨Var⟩ = ⟨Expresion⟩ ;	{ if Expresion.type != Type.type -> SDT Error (Type Mismatch); else: Declaration.exp = Type.type + Var.val + '=' + Expresion.value + ';' }
7	⟨Type⟩ →int	{ Type.type = 'int' }
8	⟨Type⟩ →float	{ Type.type = 'float' }
9	⟨Type⟩ →string	{ Type.type = 'string' }
10	⟨Var⟩ →id	{ Var.val = id.lexval }
11	⟨PrintStatement⟩ →print (⟨Expresion⟩) ;	{ PrintStatement.val = 'print(' + Expresion.value + ');' }
12	⟨Expresion⟩ →⟨Termino⟩ ⟨Expresion’⟩	{ Expresion'.inh = Termino.value; Expresion.value = Expresion'.syn }
13	⟨Expresion’⟩ →+ ⟨Termino⟩ ⟨Expresion’1⟩	{ Expresion'1.inh = Expresion'.inh + Termino.value; Expresion'.syn = Expresion'1.syn }
14	⟨Expresion’⟩ →- ⟨Termino⟩ ⟨Expresion’1⟩	{ Expresion'1.inh = Expresion'.inh - Termino.value; Expresion'.syn = Expresion'1.syn }
15	⟨Expresion’⟩ →ε	{ Expresion'.syn = Expresion'.inh }
16	⟨Termino⟩ →⟨Factor⟩ ⟨Termino’⟩	{ Termino'.inh = Factor.value; Termino.value = Termino'.syn }
17	⟨Termino’⟩ →* ⟨Factor⟩ ⟨Termino’1⟩	{ Termino'1.inh = Termino'.inh * Factor.value; Termino'.syn = Termino'1.syn }
18	⟨Termino’⟩ →/ ⟨Factor⟩ ⟨Termino’1⟩	{ if Factor.value == 0 -> SDT Error (Division by zero); else: Termino'1.inh = Termino'.inh / Factor.value; Termino'.syn = Termino'1.syn }
19	⟨Termino’⟩ →% ⟨Factor⟩ ⟨Termino’1⟩	{ if Factor.value == 0 -> SDT Error (Modulo by zero); else: Termino'1.inh = Termino'.inh % Factor.value; Termino'.syn = Termino'1.syn }
20	⟨Termino’⟩ →ε	{ Termino'.syn = Termino'.inh }
21	⟨Factor⟩ →constant	{ Factor.value = constant.lexval; Factor.type = 'int' }
22	⟨Factor⟩ →decimal	{ Factor.value = decimal.lexval; Factor.type = 'float' }
23	⟨Factor⟩ →literal	{ Factor.value = literal.lexval; Factor.type = 'string' }
24	⟨Factor⟩ →(⟨Expresion⟩)	{ Factor.value = Expresion.value; Factor.type = Expresion.type }

3.3. Algorithmic Description of Main Components and Classes

The conceptual improvements implemented in the main classes are described below:

1. **Enriched Error Manager (`errorHandler`):**
   Includes the new `has_lexical_error` flag to isolate unrecognized character failures. It retains the `has_parsing_error` and `has_sdt_error` flags to control the final execution status and prints diagnostics with exact line and column numbers.
2. **Enhanced Lexical Analyzer (`Lexer`):**
   - Adjusts column counting starting from **1** (instead of 0) to align with standard text editors.
   - Adds the `\r` character to ignored whitespace (`BLANK_SPACE`), preventing false errors when processing files saved with Windows line endings (`\r\n`).
   - Normalizes operators by unifying them into the pattern `[=+\-*/%]`.
3. **Unified Top-Down Parser (`ParserTreeBuilder`):**
   - Implements a clean LL(1) recursive descent parser. It unifies `int`, `float`, and `string` declarations into a single `declaration()` method, supported by the auxiliary methods `type_()` and `var()`.
   - Simplifies expression parsing across the `expresion()`, `termino()`, and `factor()` methods. The parser validates only the **syntactic form** of the expression, delegating **data type** verification entirely to the semantic phase.
4. **Semantic Analyzer and SDT (`SemanticAnalyzer`):**
   Applies the *Visitor* pattern over the AST nodes (`ProgramNode`, `DeclarationNode`, `PrintStatementNode`, `BinaryOpNode`, `ValueNode`). For each node:
   - Converts and verifies types using the auxiliary function `type_name(val)`.
   - Detects and interrupts incompatible assignment errors (*Type Mismatch*), division or modulo by zero, and arithmetic operations involving strings.
   - Maintains a detailed trace of attribute synthesis (`Factor{F.value = ..., F.type = ...}`).

3.4. Theoretical Justification of Output Scenarios and Error Handling

- **Scenario 1: `Parsing Success! SDT Verified!`**
  Theoretical Justification: The token sequence is fully derived by the syntactic rules of the top-down parser (the entire input is traversed without triggering `has_parsing_error`), and attribute evaluation in the SDT detects no type or mathematical violations (`has_sdt_error = False`).

- **Scenario 2: `Parsing error... SDT error...`**
  Theoretical Justification: The token sequence violates the Context-Free Grammar (e.g., an unexpected token like `int float = 2323.32;`). Upon failing syntactic matching, the parser cannot build the AST and halts execution, marking both the syntactic and semantic phases as failed.
- **Scenario 3: `Parsing Success! SDT error...`**
  Theoretical Justification: The token string is 100% syntactically valid according to the 24 grammar rules (yielding `Parsing Success!`), but the `SemanticAnalyzer`'s AST traversal detects a static logical violation (e.g., assigning a `float` value to an `int` variable), triggering the semantic error flag.
- **Additional Scenario: `Lexical error...`**
  Theoretical Justification: The lexical analyzer encounters an unrecognized character in the source code (e.g., `$`), halting the process prior to invoking the parsing phase.

---

4. Results

The test runs obtained by executing the program on the three required test cases are presented below.

4.1. Scenario 1: Successful Syntactic Recognition and Semantic Verification

Input Code:

```
1 int x = 10;
```

Console Output:

```
1 Parsing Success!
2 SDT Verified!
```

Description and Algorithmic Trace:

1. The `Lexer` generates the sequence: `[KEYWORD 'int', IDENTIFIER 'x', OPERATOR '=', CONSTANT '10', PUNCTUATION ';']`.
2. `ParserTreeBuilder.declaration()` recognizes `Type -> int`, `Var -> x`, `=`, and calls `expresion()`, which derives `Termino` and `Factor -> constant (10)`. It finishes with `;` and returns a `DeclarationNode`.

3. SemanticAnalyzer.visit() evaluates the expression (10, type int), verifies that it matches the declared type int, and registers the variable. With no errors present, it outputs Parsing Success! SDT Verified!.

4.2. Scenario 2: Syntactic Error Detection (*Parsing Error*)

Input Code:

```
int float = 2323.32;
```

Console Output:

```
Parsing error...
SDT error...

--- Error Details ---
Parsing Error [Line 1, Column 5]: Expected an identifier after 'int',
    ↪ found 'float'
```

Description and Algorithmic Trace:
After consuming the keyword int, the declaration() method invokes var(), which expects a token of type IDENTIFIER. However, the lexer categorized float as a KEYWORD. The match() method detects the discrepancy, calls report_error(), activates the failure flags, and halts parsing, reporting the exact location (Line 1, Column 5).

4.3. Scenario 3: Semantic Error Detection (*Type Mismatch*)

Input Code:

```
int a = 7.8;
```

Console Output:

```
Parsing Success!
SDT error...

--- Error Details ---
SDT Error [Line 1, Column 1]: Type Mismatch: variable 'a' is int, assigned
    ↪ value is float
```

Description and Algorithmic Trace:
The structure int a = 7.8; perfectly satisfies the grammar rule Declaration -> Type Var = Expresion ;, allowing the parser to complete successfully (Parsing Success!). However, upon executing SemanticAnalyzer.visit(), the type of the assigned value (float) is extracted and compared against the variable's declared type (int). Because they do not match, has_sdt_error is activated and the type mismatch detail is reported.

---

5. Conclusions

Implementing a **Top-Down LL(1)** paradigm and a decoupled architecture between syntax and semantics led to the following key technical insights and milestones:

1. **Elegance of the LL(1) Grammar:** Unifying declarations and expressions within the grammar simplified the parsing structure to just 24 deterministic productions, ensuring that the recursive descent control flow remains intuitive and maintainable.
2. **Effective Decoupling of Responsibilities:** Delegating syntactic validation exclusively to `ParserTreeBuilder` and type checking to `SemanticAnalyzer` demonstrated clear architectural benefits. This avoids duplicating grammar rules for each data type and centralizes semantic logic within the AST traversal.
3. **Quality Software Engineering:** Incorporating corrections in column counting (starting at 1), compatibility with Windows line endings, and clear error flag management ensures robust software that complies with all required standard specifications.